

Introdução ao Conda e GitHub

Gerenciando Ambientes em Python

Professora Carolina Ribeiro Xavier
CCOMP - UFSJ

Objetivos da Aula

- ▶ Compreender o que é o Conda.
- ▶ Aprender a criar, ativar e desativar ambientes.
- ▶ Instalar pacotes utilizando o Conda.

O que é o Conda?

► Gerenciador de Pacotes e Ambientes:

- Ferramenta para instalar, atualizar e gerenciar bibliotecas e dependências.
- Permite criar ambientes isolados para diferentes projetos, evitando conflitos entre pacotes.

Diferença entre Conda e pip para instalação

▶ **Conda:**

- ▶ Gerencia pacotes e ambientes de forma abrangente.
- ▶ Funciona com pacotes de diversas linguagens (Python, R, etc.).
- ▶ Resolve dependências complexas automaticamente.

▶ **pip:**

- ▶ Gerenciador de pacotes exclusivo para Python.
- ▶ Foca apenas em bibliotecas Python e suas dependências.
- ▶ Menos eficaz em resolver dependências de outros tipos de pacotes.

Importância do Gerenciamento de Ambientes

▶ **Isolamento:**

- ▶ Permite trabalhar em projetos com requisitos diferentes sem conflitos.
- ▶ Evita que alterações em um projeto afetem outros.

▶ **Reprodutibilidade:**

- ▶ Facilita a replicação de ambientes de desenvolvimento e produção.
- ▶ Torna mais simples compartilhar projetos com outros usuários.

▶ **Facilidade de Manutenção:**

- ▶ Atualizações de pacotes podem ser feitas de forma controlada.
- ▶ Possibilidade de reverter a versões anteriores, se necessário.

Instalação do Anaconda ou Miniconda

▶ **Anaconda:**

- ▶ Distribuição completa que inclui o Conda, pacotes populares e ferramentas de ciência de dados.
- ▶ Ideal para quem deseja um ambiente pronto para uso.
- ▶ **Download:**
`https://www.anaconda.com/products/distribution`

▶ **Miniconda:**

- ▶ Versão mínima do Anaconda, contendo apenas o Conda e suas dependências.
- ▶ Permite instalar apenas os pacotes necessários, economizando espaço.
- ▶ **Download:**
`https://docs.conda.io/en/latest/miniconda.html`

Links Úteis e Documentação Oficial

▶ **Documentação do Conda:**

- ▶ Tutorial e guia completo para usuários:

<https://docs.conda.io/projects/conda/en/latest/user-guide/index.html>

▶ **Anaconda Navigator:**

- ▶ Interface gráfica para gerenciamento de pacotes e ambientes.
- ▶ Documentação:

<https://docs.anaconda.com/anaconda/navigator/>

▶ **Comunidade e Suporte:**

- ▶ Fóruns e suporte comunitário:

<https://forum.anaconda.com/t/welcome-to-the-anaconda-community/7>

Criando um Ambiente

- ▶ Comando: `conda create --name nome_do_ambiente python=version`

Ativando e Desativando Ambientes

- ▶ Ativar: `conda activate nome_do_ambiente`
- ▶ Desativar: `conda deactivate`

Listando Ambientes

- ▶ Comando: `conda env list`
- ▶ Visualização dos ambientes existentes.

Instalando Pacotes

- ▶ Comando: `conda install nome_do_pacote`
- ▶ Instalação de conjunto de pacotes: `conda install - - file requirements.txt`
- ▶ `conda env create -f environment.yml` (resolve dependências)
- ▶ `conda list - -export > requirements.txt` (exporta as bibliotecas)
- ▶ `conda env export - -from-history > environment.yml` (controla as versões e dependências)

Atualizando e Removendo Pacotes

- ▶ Atualizar: `conda update nome_do_pacote`
- ▶ Remover: `conda remove nome_do_pacote`

Dicas e Boas Práticas

- ▶ Manter ambientes organizados.
- ▶ Usar ambientes separados para diferentes projetos.

Introdução ao Git

- ▶ Git é o sistema de controle de versões mais usado atualmente.
- ▶ Permite gerenciar o histórico de versões de um projeto de software.
- ▶ Conceitos como repositórios locais e remotos facilitam a colaboração entre desenvolvedores.

Comandos init e clone

Init:

- ▶ Cria um repositório Git vazio.
- ▶ Exemplo: `git init`
- ▶ Após o `init`, o Git começa a rastrear mudanças nos arquivos.

Clone:

- ▶ Clona um repositório existente (ex: GitHub).
- ▶ Exemplo: `git clone https://github.com/user/repo.git`
- ▶ Útil para começar a trabalhar em projetos existentes.

Exemplo prático:

- ▶ Clone um repositório aberto do GitHub: `git clone https://github.com/public/repo.git`

Comando commit

- ▶ `git commit` cria um snapshot dos arquivos rastreados.
- ▶ Commits devem ser realizados periodicamente para registrar mudanças importantes.
- ▶ Exemplo: `git commit -m "Adicionando nova funcionalidade"`

Exemplo prático:

- ▶ Crie um arquivo `main.py`, adicione código Python e faça um commit: `git add main.py`, depois `git commit -m "Criando script principal"`.

Comando add

- ▶ `git add` move arquivos para a área de stage (ainda não está no seu projeto).
- ▶ Apenas arquivos no stage são incluídos no próximo commit.
- ▶ Exemplo: `git add arq1.txt`

Exemplo prático:

- ▶ Modifique o `main.py` e adicione-o novamente ao stage: `git add main.py`. Para efetivar a mudança, faça o commit.

Comandos status, diff log

Status:

- ▶ Mostra o estado atual do repositório.
- ▶ Exemplo: `git status` (verificar arquivos modificados, não rastreados, etc.)

Diff:

- ▶ Exibe as diferenças entre o diretório de trabalho e o stage.
- ▶ Exemplo: `git diff`

Log:

- ▶ Lista o histórico de commits.
- ▶ Exemplo: `git log` (use `git log --oneline` para uma visualização compacta).

Exemplo prático:

- ▶ Altere o `main.py` e use `git diff` para ver as mudanças antes de fazer o commit.

Comandos push e pull

- ▶ **Push:** Envia os commits do repositório local para o remoto.
- ▶ Exemplo: `git push origin main`

Exemplo prático:

- ▶ Faça um `git push origin main` para enviar suas mudanças para o repositório GitHub.

Pull:

- ▶ Atualiza o repositório local com mudanças do repositório remoto.
- ▶ Exemplo: `git pull` (combina fetch e merge).

Conflitos de Merge

- ▶ Conflitos de merge ocorrem quando dois desenvolvedores modificam o mesmo trecho de código.
- ▶ O Git marca os trechos conflitantes para que o desenvolvedor escolha qual código manter.
- ▶ Exemplo de conflito: <<<<<< HEAD ... ===== ...
>>>>>> commitID

Exemplo prático:

- ▶ Simule um conflito de merge modificando o mesmo arquivo em duas máquinas diferentes e tente fazer o push.

Sobre o Git

- ▶ Git é uma ferramenta essencial para o controle de versões.
- ▶ Domine os comandos básicos antes de avançar para os comandos complexos.
- ▶ Use para manter seu portfólio.

Tarefa proposta

Para exercitar mais a programação em python e testar o uso das ferramentas vistas hoje, vamos fazer uma implementação simples usando um conjunto de bibliotecas em python.